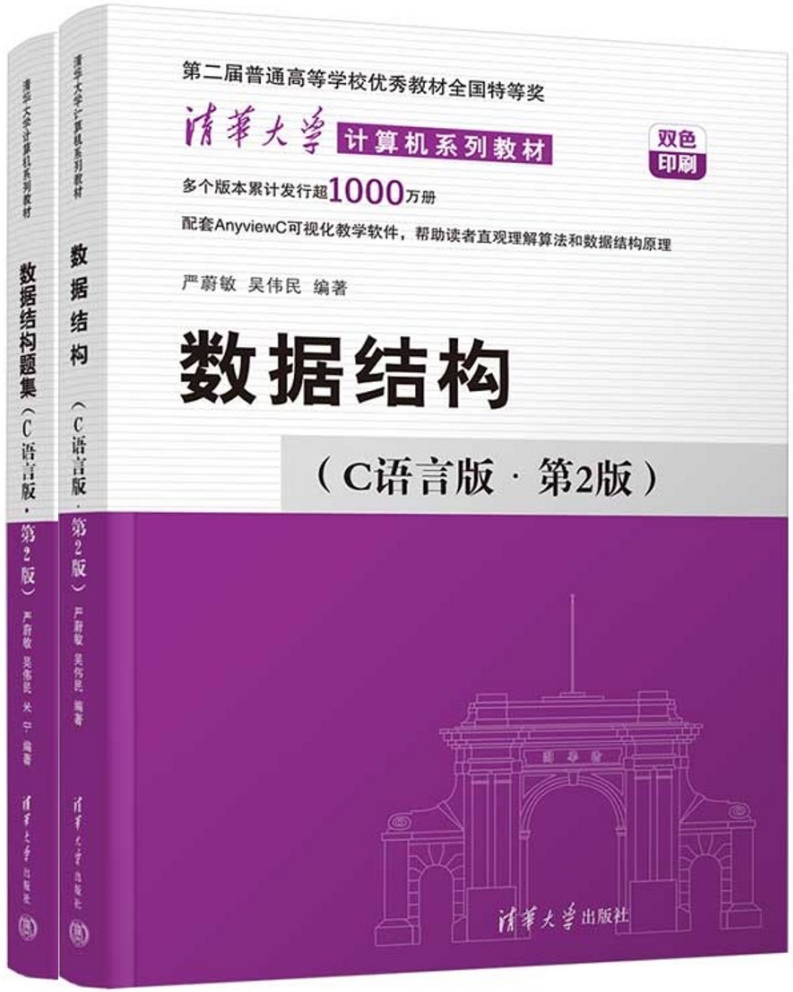


清華大学出版社
TSINGHUA UNIVERSITY PRESS

第10章

内部排序

算法10.1 直接插入排序

插入类和选择类排序的排序过程是一个逐步扩大有序子表的过程。

直接插入排序是最简单的插入类排序。

每一趟把无序子表的第一个记录插入到有序子表，需要为该记录找到并腾出该插入位置。

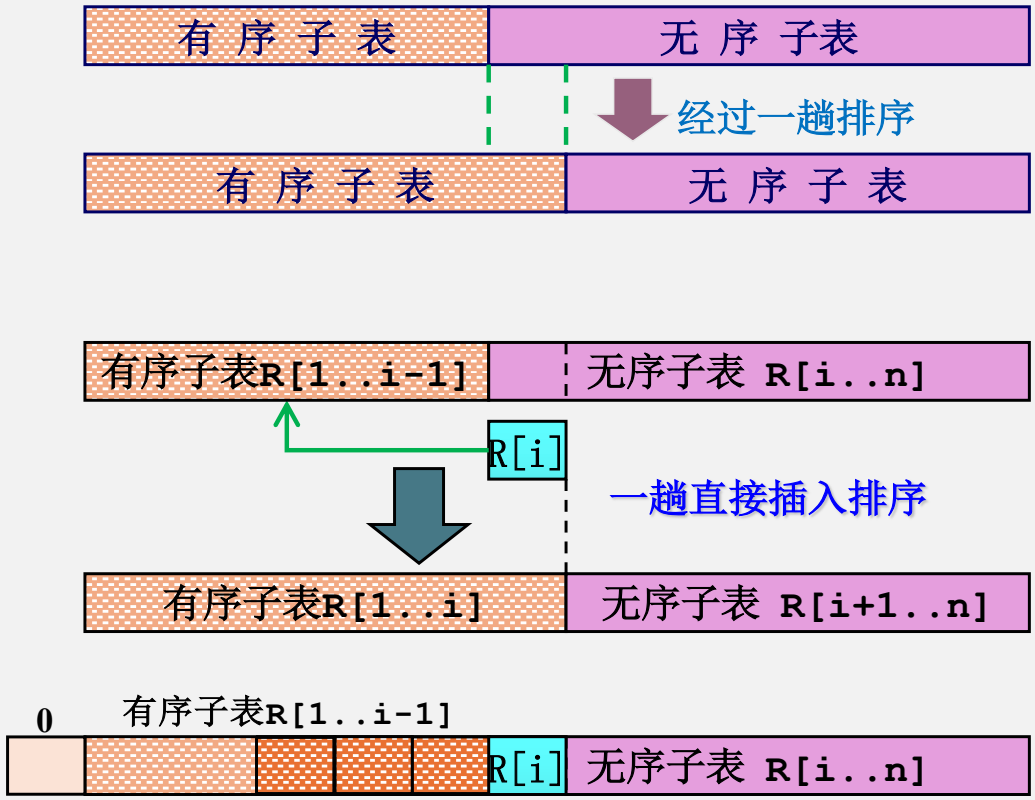
找位置可在有序子表进行正向或反向扫描。

腾位置则只能从后往前，逐个记录后移。

可以从后往前，找位和腾位同时进行。

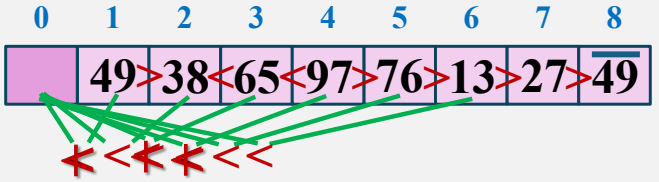
利用0号单元暂存要插入的记录，以便较大记录后移，还可作为找位置过程的“哨兵”。

如果找到0号单元时，就是记录和自己比较，因“不小于”而结束查找，插入记录比有序子表的记录都小，插入到1号单元。



算法10.1 直接插入排序

综所上述，实现直接插入排序的算法10.1设计如下。
对于长度为n的顺序表，进行n-1趟的插入排序。
右上方的示例是一个长度为8的线性表，有两个49，
后一个顶上加了横杠，以示区分，可观察排序后，
两个49的相对次序是否改变？




```
void InsertSort(SqList L) {   // 算法10.1
   // 对顺序表L作直接插入排序。
   for (int i=2; i<=L->len; ++i)   // 初始有序子表仅L->r[1]，共L->Len-1趟
      if (LT(L->r[i], L->r[i-1])) {   // "<"时，需将L->r[i]插入有序子表
         L->r[0] = L->r[i];          // 复制为哨兵
         L->r[i] = L->r[i-1];        // 有序子表的表末元素后移
         for (int j=i-2;  LT(L->r[0], L->r[j]);  --j)
            L->r[j+1] = L->r[j];     // 记录后移
         L->r[j+1] = L->r[0];        // 插入到正确位置
      }
}
```

清華大學出版社
TSINGHUA UNIVERSITY PRESS

算法10.3 表插入排序和记录重排

问题：如何将直接插入排序中的记录移动次数从$O(n^2)$降到$O(n)$的级别？

—— 存储结构采用**链表**，而不是**顺序表**。

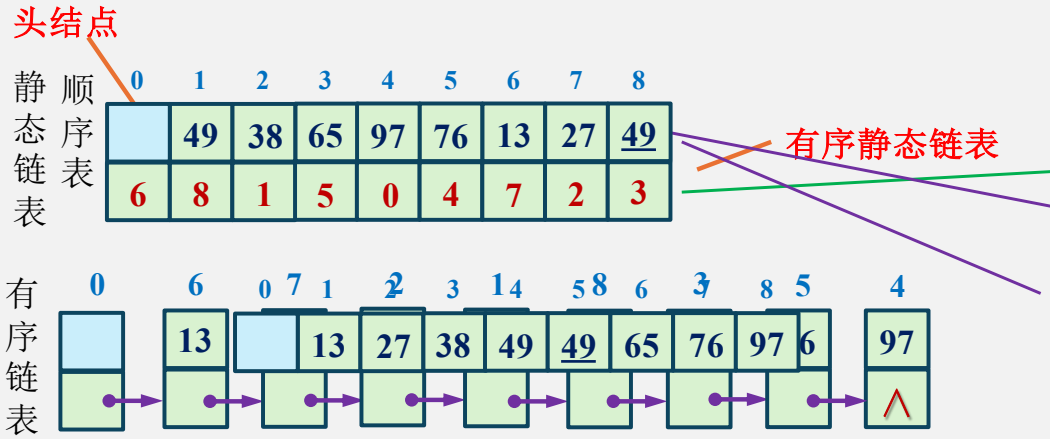
但是，排序是为了得到记录的有序序列，便于进行频繁的**折半查找**。

办法：不用动态链表，而用**静态链表**。

静态链表：由存储记录的顺序表和附加的指针数组构成。

所谓指针，就是数组单元的下标，是一个整数值。

利用这个附加的指针数组来进行插入排序的方法，就称为**表插入排序**。



表插入排序分两步：

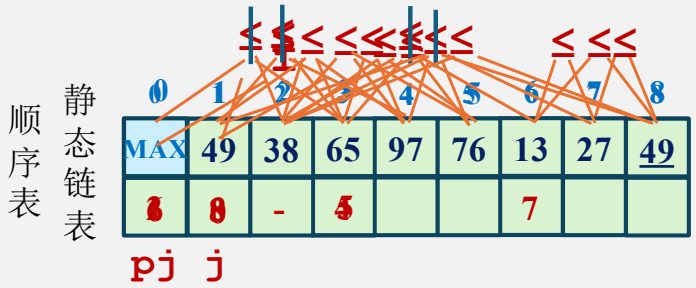
（1）静态链表插入排序；

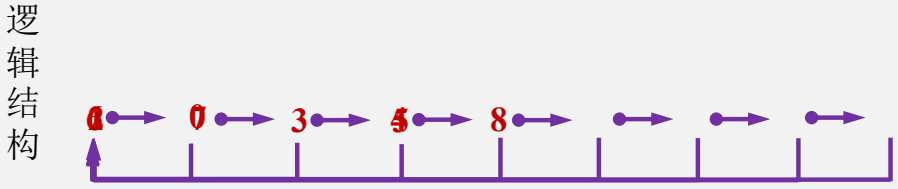
（2）按序重排顺序表的记录。（算法10.3）

算法10.3和10.4 表插入排序和记录重排

表插入排序的过程如下：

1. 构造只含一个记录的有序循环静态链表；
2. 从 i=2 起，将记录逐个插入该有序链表；
3. 根据指针值所指将记录序列调整为一个有序序列。（算法10.4）






```
void InsertSort_SL(SLinkList SL) { // 算法10.3，表插入排序
  SL->r[0].rc.key = INT_MAX; //初始化头结点，置哨兵
  SL->r[0].next=1;      // 头结点指向首个记录
  SL->r[1].next=0;      // 首个记录指回头结点，构成循环链表
  for(i=2; i<=SL->len; i++) { // 第i个记录插入有序子表
    pj=0;  j=SL->r[0].next;    // 从首个记录开始查找插入位置
    while(SL->r[j].rc <= SL->r[i].rc) { // 比较关键字
      pj = j;  j = SL->r[j].next; // 指示有序子表下一记录
    }
    SL->r[i].next = j;    // 插入第i个记录
    SL->r[pj].next = i;
  }
  Arrange(SL);      // 调用算法10.4  按序重排SL
}
```

算法10.3和算法10.4 表插入排序和记录重排

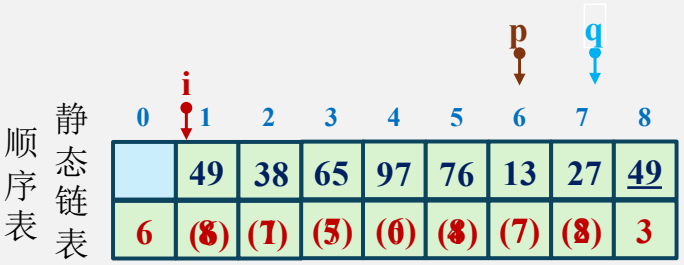
清華大學出版社 TSINGHUA UNIVERSITY PRESS

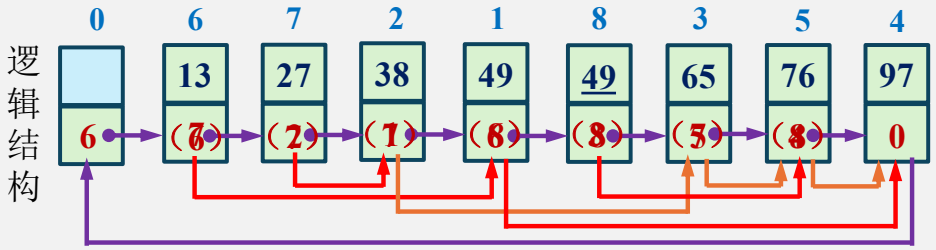
按有序静态链表重排顺序表：

链表第i个结点移到顺序表的第i位置。

图10.4初始是有序循环静态链表重排前的初态。

```
void Arrange(SLinkedList SL) {   // 算法10.4 按序重排SL
  int p = SL->r[0].next;   // p指示第一个记录的当前位置
  for(int i=1; i<SL->len; ++i) {   // SL->r[1..i-1]已有序
    while(p<i) p = SL->r[p].next; // 令p指示第i个记录当前位置
    int q = SL->r[p].next;        // q指示第i+1记录的当前位置
    if(p!=i) {     // 若p==i则第i个记录已到位，无需移动
      swap(SL->r, p, SL.r[i]);  // 通过交换让第i个记录到位
      SL->r[i].next = p;     // 到位记录指示被交换记录的新位置
    }
    p = q;          // p指示第i+1(下一个要调整到位的)记录位置
  }
}
```





清華大學出版社 TSINGHUA UNIVERSITY PRESS

算法10.5 希尔排序

这种改进的插入排序方法称为希尔排序，也称为“缩小增量插入排序”。

回顾直接插入排序，其算法时间复杂度为$O(n^2)$，主要原因是记录的每次移动都只后移1位，移动频繁，但效率低下。

如果待排序的记录序列已接近有序，时间复杂度则趋向$O(n)$。

改进思路：让记录的每次移动可后移多个位置，更接近它的排序最终位置。

实现措施：

（1）把记录序列$\{R_1,R_2,\cdots R_i\cdots R_n\}$划分为$dk$个子序列

$\{R_1,R_{1+dk},R_{1+2dk},\cdots\}$，$\{R_2,R_{2+dk},R_{2+2dk},\cdots\}$，…，$\{R_{dk},R_{dk+dk},R_{dk+2dk},\cdots\}$

（2）对每个子序列分别进行直接插入排序；

（3）减小dk，重复（1）和（2）步（称为一趟），直到最后完成一次$dk=1$，也就是对整个记录序列的直接插入排序。

dk既是子序列数，也是子序列中元素的“距离”，或称“增量”。

t趟的dk值构成一个递减到1的序列。可存入数组dlta[0..t-1]：

dlta[0]>dlta[1]> … >dlta[t-1]=1

	0	1	2	3	4	5	6	7	8	9	10
dk=5		49	38	65	97	76	13	27	49	55	04

例如，10个记录的序列，

$dk=5$，划分为5个子序列。用颜色区分。

对第1个子序列进行直接插入排序：

13<49，13移到0号单元，

49后移到13的位置，——后移了dk个位置

再把13插入到49原来位置。

同样，其他子序列，只要有记录后移，

每次都后移k个位置。

可能的疑问：

既然最后一次就是对整个记录序列的直接插入排序，前面的多次划分子序列以及分别插入排序，不是花费更多的时间吗？

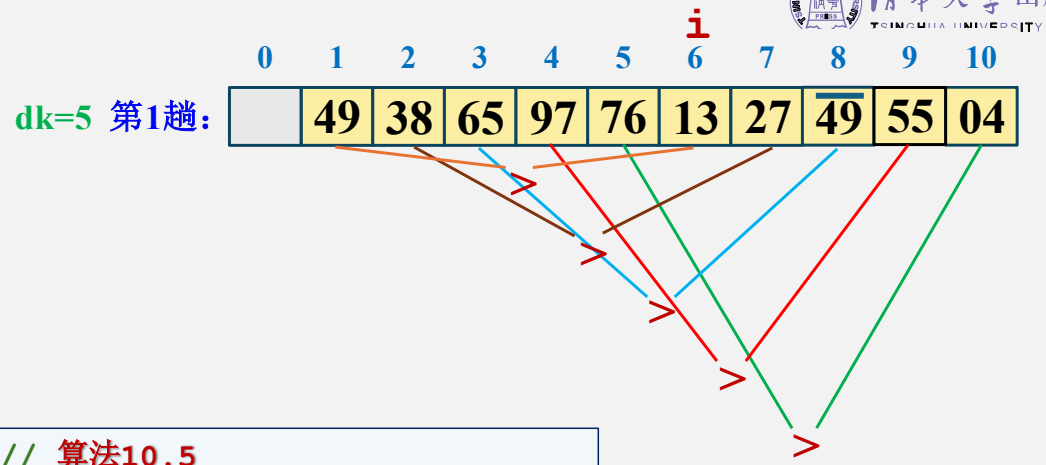
——最后一趟前，序列已“接近”有序

时间复杂度可以降低到$O(n^{3/2})$

算法10.5 希尔排序

算法10.5的函数 ShellInsert 是增量为dk的一趟希尔排序。该趟将记录划分为dk个子序列。

可与算法10.1作对比，如果说直接插入排序是dk=1的，那么算法10.4就是用dk替换算法10.1中相应的1而得到的。

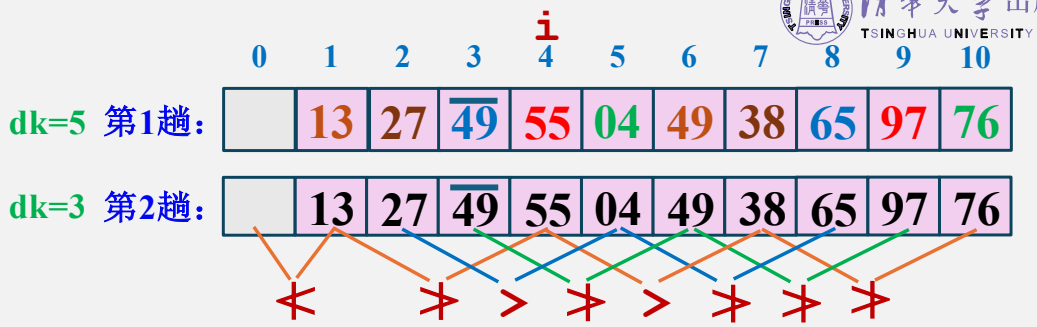


5

```
void ShellInsert(SqList L, int dk) {  // 算法10.5
  // 对顺序表L作一趟希尔插入排序。本算法对算法10.1作了以下修改:
  //     1. 本趟将记录划分为dk个子序列;
  //     2. 前后记录位置的增量（距离）是dk，而不是1;
  //     3. r[0]只是暂存单元，不是哨兵。当j<=0时，插入位置已找到。
  for (int i=dk+1; i<=L->len; ++i)
    if (LT(L.r[i].key, L.r[i-dk].key)) { // L.r[i]较小则插入有序增量子表
      L->r[0] = L->r[i];                 // 先暂存在L->r[0]
      L->r[i] = L->r[i-dk];
      for (int j=i-2*dk; j>0 && LT(L->r[0], L->r[j]); j-=dk)
        L->r[j+dk] = L->r[j];    // 记录后移，查找插入位置
      L->r[j+dk] = L->r[0];      // 插入
    }
}
```

算法10.5 希尔排序

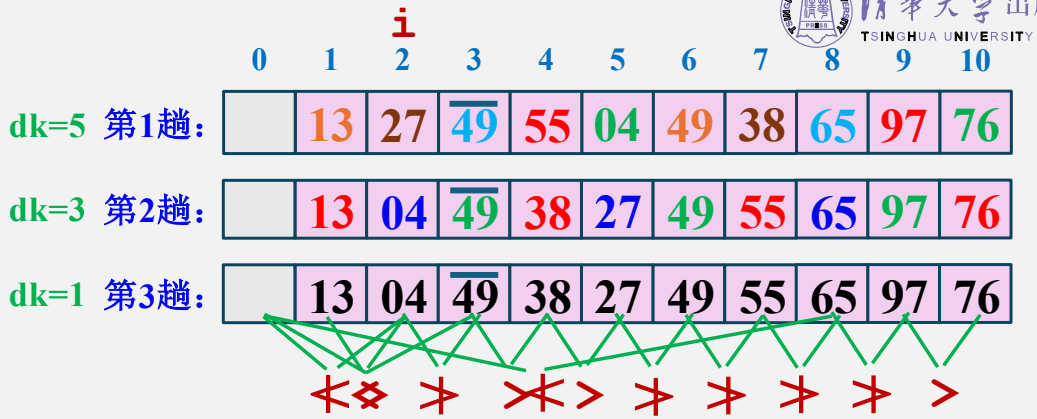
接着是增量为3的第2趟




```
void ShellInsert(SqList L, int dk) {  // 算法10.5
  // 对顺序表L作一趟希尔插入排序。本算法对算法10.1作了以下修改：
  //     1. 本趟将记录划分为dk个子序列；
  //     2. 前后记录位置的增量（距离）是dk，而不是1；
  //     3. r[0]只是暂存单元，不是哨兵。当j<=0时，插入位置已找到。
  for (int i=dk+1; i<=L->len; ++i)
    if (LT(L.r[i].key, L.r[i-dk].key)) { // L.r[i]较小则插入有序增量子表
      L->r[0] = L->r[i];                 // 先暂存在L->r[0]
      L->r[i] = L->r[i-dk];
      for (int j=i-2*dk; j>0 && LT(L->r[0], L->r[j]); j-=dk)
        L->r[j+dk] = L->r[j];    // 记录后移，查找插入位置
      L->r[j+dk] = L->r[0];      // 插入
    }
}
```

算法10.5 希尔排序

希尔排序的最后一趟的增量dk必须为1。
把算法中的dk全部换回1，算法就还原回直接插入排序的算法10.1了。
我们看增量dk为1的最后一趟。




```
void ShellInsert(SqList L, int dk) {  // 算法10.5
  // 对顺序表L作一趟希尔插入排序。本算法对算法10.1作了以下修改:
  //     1. 本趟将记录划分为dk个子序列;
  //     2. 前后记录位置的增量（距离）是dk，而不是1;
  //     3. r[0]只是暂存单元，不是哨兵。当j<=0时，插入位置已找到。
  for (int i=dk+1; i<=L->len; ++i)
    if (LT(L.r[i].key, L.r[i-dk].key)) { // L.r[i]较小则插入有序增量子表
      L->r[0] = L->r[i];                 // 先暂存在L->r[0]
      L->r[i] = L->r[i-dk];
      for (int j=i-2*dk; j>0 && LT(L->r[0], L->r[j]); j-=dk)
        L->r[j+dk] = L->r[j];    // 记录后移，查找插入位置
      L->r[j+dk] = L->r[0];      // 插入
    }
}
```

算法10.5 希尔排序

清華大學出版社 TSINGHUA UNIVERSITY PRESS

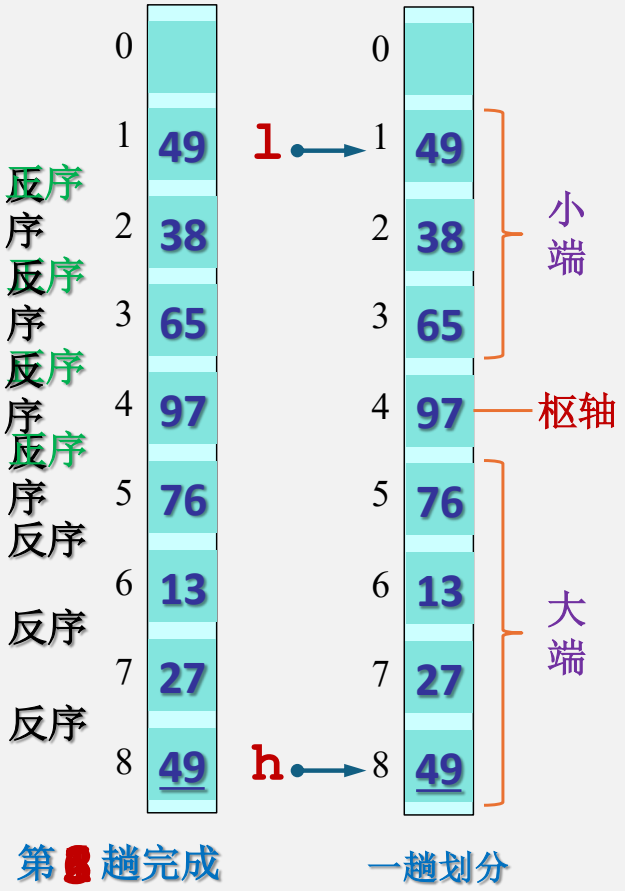
前面的各趟希尔排序就是对ShellInsert的三次调用，参数dk的实参依次是5、3、1。算法10.5提供了一个t趟希尔排序的接口。由参数d[]数组获得t个降序的增量，并用以对ShellInsert函数的t次调用。

	0	1	2	3	4	5	6	7	8	9	10
dk=5 第1趟：		13	27	49	55	04	49	38	65	97	76
dk=3 第2趟：		13	04	49	38	27	49	55	65	97	76
dk=1 第3趟：		04	13	27	38	49	49	55	65	76	97

```
void ShellInsert(SqList L, int dk) {  // 算法10.5
  // 对顺序表L作一趟希尔插入排序。本算法对算法10.1作了以下修改：
  //     1. 本趟将记录划分为dk个子序列；
  //     2. 前后记录位置的增量（距离）是dk，而不是
  //     3. r[0]只是暂存单元，不是哨兵。当j<=0时，
  for (int i=dk+1; i<=L->len; ++i)
    if (LT(L.r[i].key, L.r[i-dk].key)) {
      L->r[0] = L->r[i];
      L->r[i] = L->r[i-dk];
      for (int j=i-2*dk; j>0 && LT(L->r[0], L->r[j]); j-=dk)
        L->r[j+dk] = L->r[j];    // 记录后移，查找插入位置
      L->r[j+dk] = L->r[0];      // 插入
    }
}
```

```
void ShellSort(SqList L, int d[], int t) {
  // 算法10.5按增量序列d[0..t-1]对顺序表L作希尔排序
  for (int k=0; k<t; ++k)
    ShellInsert(L, d[k]);
    // 一趟增量为dlta[k]的插入排序
}
```

算法10.6和10.7 快速排序




```
void bubble_sort(SqList L) { // 起泡排序
  for(int i=L->len,change=TRUE; i>1 && change; --i) {
    change=FALSE;
    for(int j=1; j<i; ++j) // 一趟
      if(L->r[j]>L->r[j+1])
        { L->r[j]⇔L->r[j+1]; change=TRUE; }
  }
} // O(n²)的 ——元素每次只移动1位，龟速！但有1元素到位
```

第六趟没有交换元素，change没有改变，排序结束！

一趟的改进——元素大跨度移位，
令到位元素之前的元素全小于它，
之后的元素全不小于它。

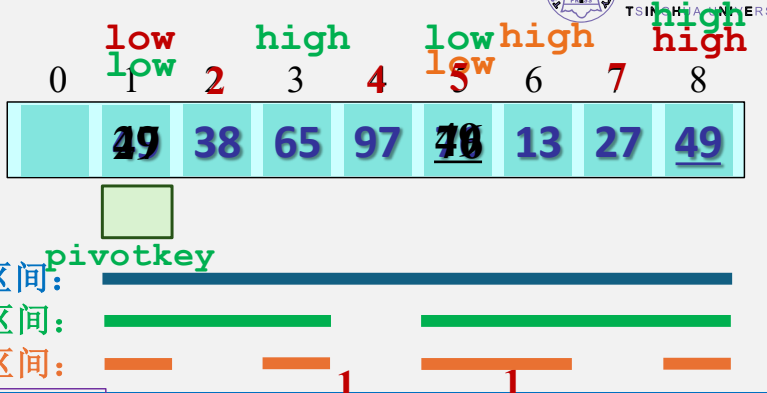
```
int Partition(SqList L, int low, int high) { // 算法10.6(b)
  // 交换顺序表L中子序列L->r[low..high]的记录，使枢轴记录到位，
  // 并返回其所在位置，此时，在它之前（后）的记录均不大（小）于它
  L->r[0] = L->r[low]; pivotkey = L->r[low].key; // 枢轴记录
  while (low<high) {   // 从表的两端交替地向中间扫描
    while (low<high && L->r[high].key>=pivotkey) --high;
    L->r[low] = L->r[high];  // 将比枢轴记录小的记录移到低端
    while (low<high && L->r[low].key<=pivotkey) ++low;
    L->r[high] = L->r[low];  // 将比枢轴记录大的记录移到高端
  }
  L->r[low] = L->r[0];    // 枢轴记录到位
  return low;             // 返回枢轴位置
}
```

算法10.6和10.7 快速排序

算法10.7是如何表达上述基于划分递归分解分段排序的呢？

对示例调用算法，演示快速排序过程

一般应用是对整个顺序表排序，不需要指定区间。
算法10.8给出了快速排序的规范接口。




```
void QSort(SqList L, int low, int high) { //算法10.7
  // 对顺序表L中的子序列L->r[low..high]进行快速排序
  if (low < high) {   // 长度大于1
    pivotloc = Partition(L, low, high);  // 将L->r[low..high]一分为二，pivotloc是枢轴位置
    QSort(L, low, pivotloc-1);   // 对低子表递归排序
    QSort(L, pivotloc+1, high);  // 对高子表递归排序
  }
}
```

```
void QuickSort(SqList L) {
  // 算法10.7 对顺序表L作快速排序
  QSOrt(L, 1, L->len);
}
```

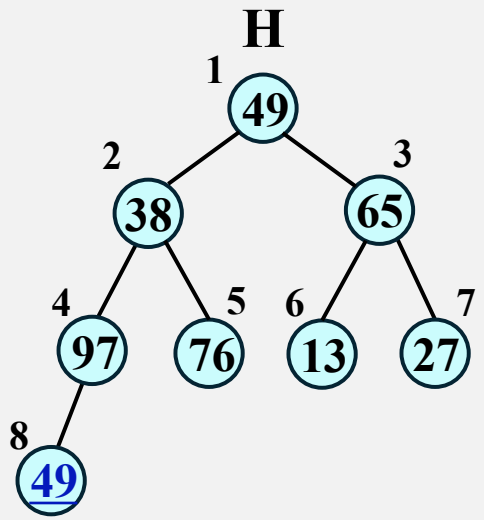
算法10.9 堆排序 —— 用6.4节的堆进行排序

清華大學出版社 TSINGHUA UNIVERSITY PRESS

```
typedef SqList Heap;   // 堆为顺序表类型
void HeapSort(SqList L) {
  // 对顺序表 L 堆排序
  int len = L->len;          // 保存表长
  Heap H = MakeHeap(L);      // 建成大顶堆
```

```
} // HeapAdjust
```

1	2	3	4	5	6	7	8
49	38	65	97	76	13	27	49



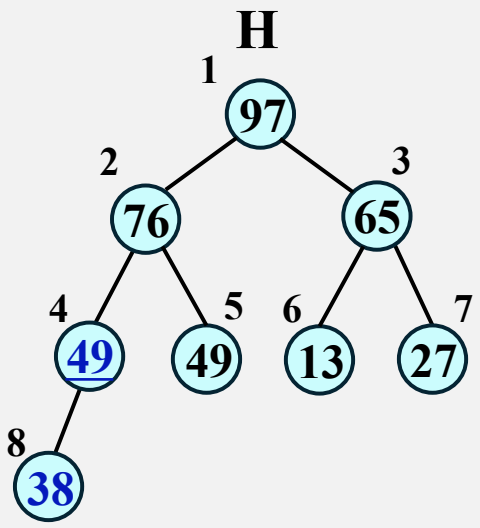
算法10.9 堆排序(续) ——用6.4节的堆进行排序

清華大學出版社 TSINGHUA UNIVERSITY PRESS

```
typedef SqList Heap;   // 堆为顺序表类型
void HeapSort(SqList L) {
  // 对顺序表 L 堆排序
  int len = L->len;          // 保存表长
  Heap H = MakeHeap(L);      // 建成大顶堆
  while (H->len>1) {
    swapElem(H, 1, H->len); // 堆顶末交换
    H->len--;               // 堆长减1
    SiftDown(H, 1);         // 调堆
  }
  L->len = len;   // 恢复表长
} // HeapAdjust
```

H->len

1	2	3	4	5	6	7	8
97	76	65	49	49	13	27	38



算法10.10和11 归并排序（递归）

清華大學出版社 TSINGHUA UNIVERSITY PRESS

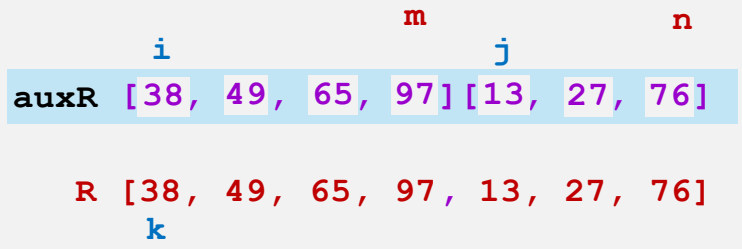
归并排序(Merging Sort)是又一类不同的排序方法。“归并”的含义是将两个或两个以上的有序表组合成一个新的有序表。它的实现方法早已为读者所熟悉，无论是顺序存储结构还是链表存储结构，都可在$O(m+n)$的时间量级上实现。

假设两个有序表的长度分别为m和n。

2-路归并排序中的核心操作是将一维数组中前后相邻的两个有序序列归并为一个有序序列，其算法（类似于算法2.7）如**算法10.10**所示。

```
RcdType *auxR;
void Merge(RcdType R[], int i, int m, int n){
  // 将有序的R[i..m]和R[m+1..n]归并为有序的R[i..n]
  int i, j;
  for(j=i; j<=n; ++j) auxR[j]=R[j]; //复制到辅助数组
  for(j=m+1,k=i; i<=m && j<=n; ++k) //复制auxR到R
    if(auxR[i].key<=auxR[j].key) R[k]=auxR[i++];
    else R[k]=auxR[j++];
  // 将剩余auxR[i..m]复制到R
  while(k<=n && i<=m) R[k++]=auxR[i++];
  // 将剩余的auxR[j..n]复制到R
  while(k<=n && j<=n) R[k++]=auxR[j++];
} // Merge 算法10.10
```



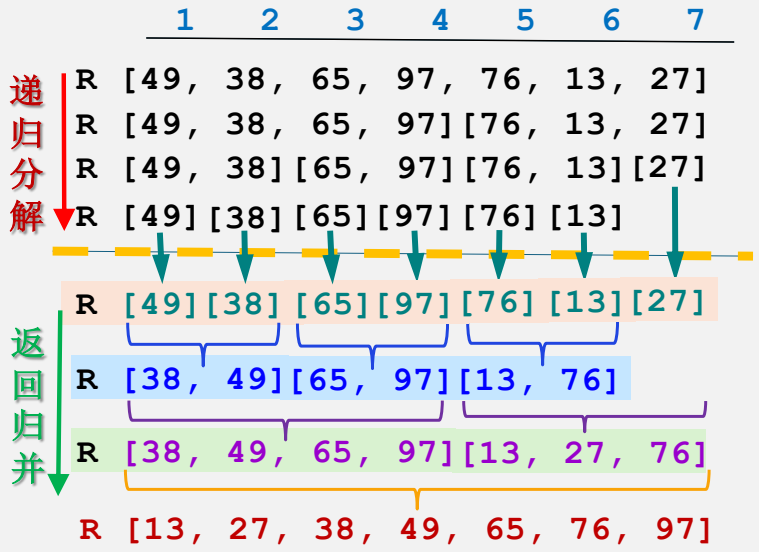


算法10.10和11 归并排序（递归）

```
void MSort(RcdType R[], int s, int t) {
  if (s<t) {
    int m = (s+t)/2;   // 求中点
    MSort(R, s, m);    // 归并排序R[s..m]
    MSort(R, m+1, t); // 归并排序R[m+1..t]
    Merge(R, s, m, t); // 归并
  }
} // Msort 算法10.11
```

```
void MergeSort(SqList L) {   // 算法10.11
  // 对顺序表L作归并排序
  auxR=(RcdType*)calloc(L->len+1,sizeof(RcdType));
  MSort(L->r, 1, L->len);
  free(auxR);
}
```

```
RcdType *auxR;
void Merge(RcdType R[], int i, int m, int n) {
  // 将有序的R[i..m]和R[m+1..n]归并为有序的R[i..n]
  int i, j;
  for(j=i; j<=n; ++j) auxR[j]=R[j]; //复制到辅助数组
  for(j=m+1,k=i; i<=m && j<=n; ++k) //复制auxR到R
    if(auxR[i].key<=auxR[j].key) R[k]=auxR[i++];
    else R[k]=auxR[j++];
  // 将剩余auxR[i..m]复制到R
  while(k<=n && i<=m) R[k++]=auxR[i++];
  // 将剩余的auxR[j..n]复制到R
  while(k<=n && j<=n) R[k++]=auxR[j++];
} // Merge 算法10.10
```



链式基数排序

链式基数排序是在链表的结构上实现基数排序的算法。

类似于表插入排序，在存储结构中每个表结点添加一个**next**指针，将存储记录的顺序表改造为一个静态链表。并通过修改结点的**next**指针，来实现基数排序中的分配和收集。

```
typedef struct {
  KeysType keys[MAX_NUM_OF_KEY]; // 关键字
  int next;                      // 链指针
  InfoType otheritems;    // 其他数据项
} SLCell;       // 静态链表的结点类型
typedef struct {
  SLCell *r;  // 静态链表结点的存储空间，r[0]为头结点
  int  keynum;  // 记录的关键字个（位）数
  int  recnum;  // 静态链表当前长度
} *SLList;        // 静态链表类型
```

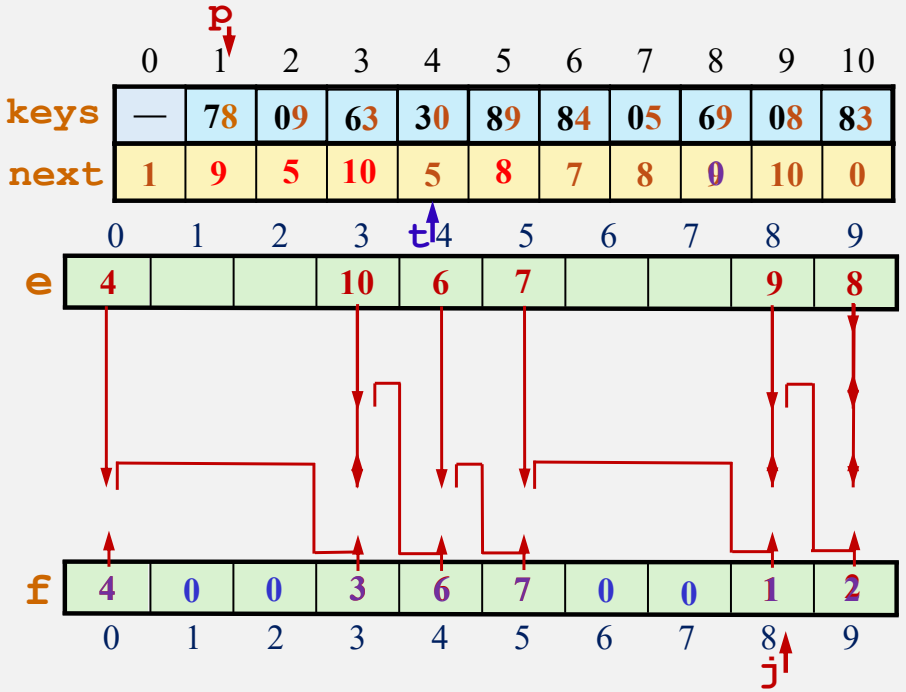
```
void RadixSort(SLList L) {   // 算法10.15
  // L是采用静态链表表示的顺序表。
  // 对L作基数排序，使得L成为按关键字自小到大的有序静态链表，L->r[0]为头结点。
  int i;  ArrType f, e;
  for (i=1; i<L->len; ++i) L->r[i-1].next = i;
  L->r[L->len].next = 0;          // 将L改造为静态链表
  for (i=0; i<L->keynum; ++i) {  // 按最低位优先依次对各关键字作分配和收集
    Distribute(L->r, i, f, e);   // 第i趟分配。调用算法10.13
    Collect(L->r, i, f, e);      // 第i趟收集。调用算法10.14
  }
}
```

链式基数排序

```
void Distribute(SLCell r[], int i,
                ArrType f, ArrType e) {
  for(j=0; j<RADIX; ++j) f[j] = 0;
  for(p=r[0].next; p; p=r[p].next) {
    j = r[p].keys[i]-'0'; //映射到0..RADIX-1
    if(!f[j]) f[j] = p;
    else r[e[j]].next = p;
    e[j] = p;
  }
} // 算法10.13 按第i个关键字分配
```

```
void Collect(SLCell r[], int i,
                 ArrType f, ArrType e) {
  for(j=0; !f[j]; j++);
  r[0].next = f[j];  t = e[j];
  while(j<RADIX) {
    for(j++; j<RADIX && !f[j]; j++);
    if(j<RADIX)
      { r[t].next = f[j];  t = e[j]; }
  }
  r[t].next = 0;
} // 算法10.14 按第i个关键字收集
```

待排关键字序列
初始化链表



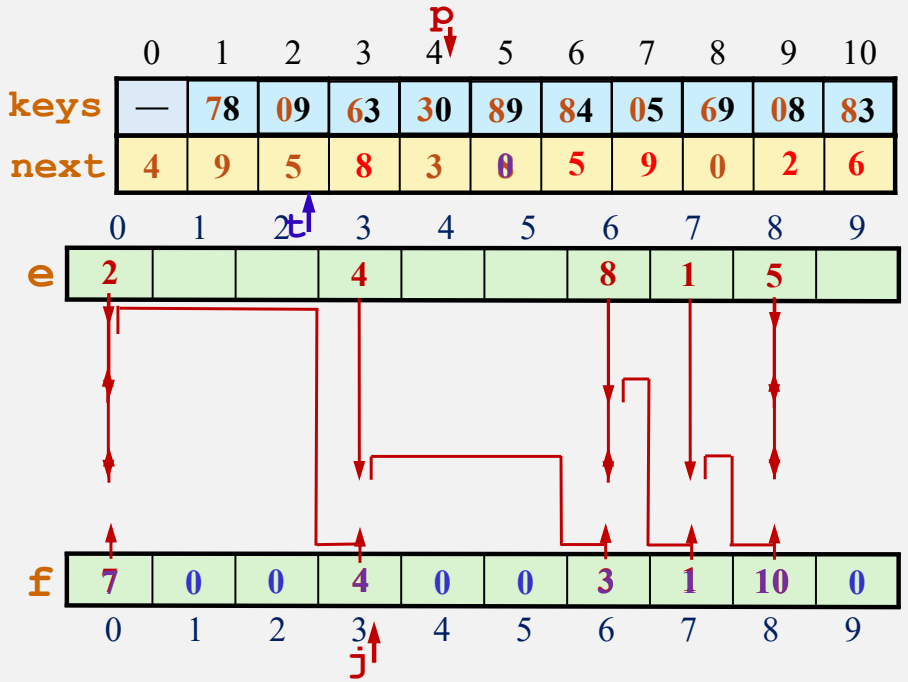
进行第一次分配

进行第一次收集

链式基数排序

```
void Distribute(SLCell r[], int i,
                ArrType f, ArrType e) {
  for(j=0; j<RADIX; ++j) f[j] = 0;
  for(p=r[0].next; p; p=r[p].next) {
    j = r[p].keys[i]-'0';
    if(!f[j]) f[j] = p;
    else r[e[j]].next = p;
    e[j] = p;
  }
} // 算法10.13 按第i个关键字分配
```

```
void Collect(SLCell r[], int i,
             ArrType f, ArrType e) {
  for(j=0; !f[j]; j++);
  r[0].next = f[j];  t = e[j];
  while(j<RADIX) {
    for(j=j+1; j<RADIX && !f[j]; j++);
    if(j<RADIX)
      { r[t].next = f[j];  t = e[j]; }
  }
  r[t].next = 0;
} // 算法10.14 按第i个关键字收集
```



进行第二次分配

进行第二次收集

对顺序表进行按序记录重排，可参见**算法10.3**